Exception:

An exception (or exceptional event) is a problem that arises during the execution of a program. When an **Exception** occurs the normal flow of the program is disrupted and the program/Application terminates abnormally, which is not recommended, therefore, these exceptions are to be handled.

An exception can occur for many different reasons. Following are some scenarios where an exception occurs.

- A user has entered an invalid data.
- A file that needs to be opened cannot be found.
- A network connection has been lost in the middle of communications or the JVM has run out of memory.

Some of these exceptions are caused by user error, others by programmer error, and others by physical resources that have failed in some manner.

A Java exception is an object that describes an exceptional (that is, error) condition that has occurred in a piece of code. When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error. That method may choose to handle the exception itself, or pass it on. Either way, at some point, the exception is caught and processed.

Exceptions can be generated by the Java run-time system, or they can be manually generated by your code. Java exception handling is managed via five keywords: **try, catch, throw, throws, and finally**.

If an exception occurs within the try block, it is thrown. Your code can catch this exception (using catch) and handle it in some rational manner. System-generated exceptions are automatically thrown by the Java runtime system. To manually throw an exception, use the keyword throw. Any exception that is thrown out of a method must be specified as such by a throws clause. Any code that absolutely must be executed after a try block completes is put in a finally block.

This is the general form of an exception-handling block:

```
try {
        // block of code to monitor for errors
}
catch (ExceptionType1 exOb) {
        // exception handler for ExceptionType1
}
catch (ExceptionType2 exOb) {
        // exception handler for ExceptionType2
}
// ...
finally {
        // block of code to be executed after try block ends
}
```

Here, *ExceptionType* is the type of exception that has occurred.

SURESH YADLAPATI

There are basically two models of exception handling.

1. Termination Model – JAVA follows termination model.
2. Resumption model

1. Termination Model: In termination model, when the method encounters an exception, further processing in that method is terminated and control is transferred from the point where an exception occurs to the point where its nearest matching exception handler (i.e., the catch block) is located.

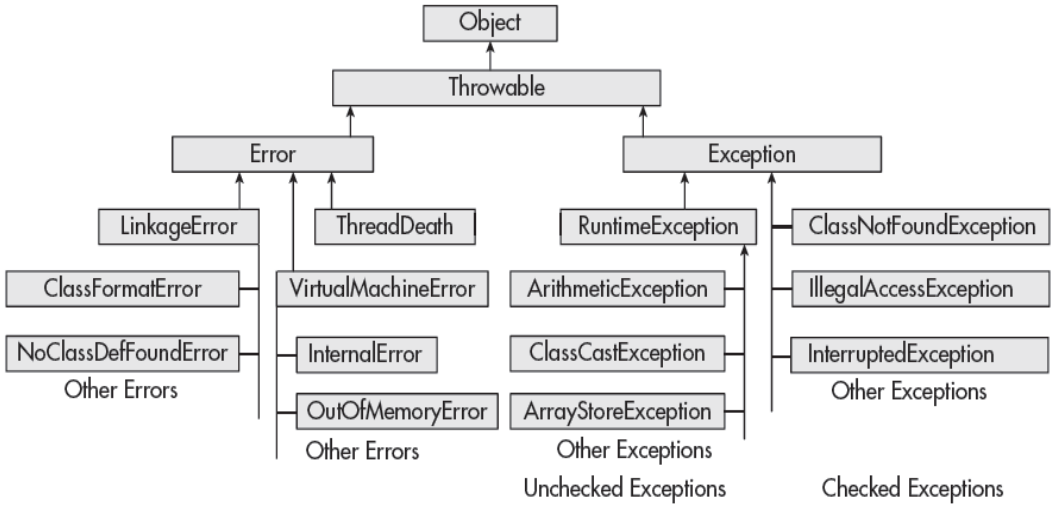
2. Resumption Model: In resumption model, the exception handler tries to rectify the exception situation and resume the program.

Hierarchy of Standard Exception Classes:

- In Java, exceptions are instances of classes derived from the class Throwable which in turn is derived from class Object.
- Whenever an exception is thrown, it implies that an object is thrown.
- Only objects belonging to class that is derived from class Throwable can be thrown as exceptions.
- The next level of derived classes comprises two classes: the class Error and class Exception. Error class involves errors that are mainly caused by the environment in which an application is running.
    - All errors in Java happen during runtime.
- Exception class represents exceptions that are mainly caused by the application itself.
- The subclasses of Exception class are broadly subdivided into two categories. **Unchecked exceptions:** These are subclasses of class RuntimeException, derived from Exception class.
    - For these exceptions, the compiler does not check whether the method that throws these exceptions has provided any exception handler code or not.

  Checked exceptions: These are direct subclasses of the Exception class and are not subclasses of the class RuntimeException. These are called so because the compiler ensures (checks) that the methods that throw checked exceptions deal with them.

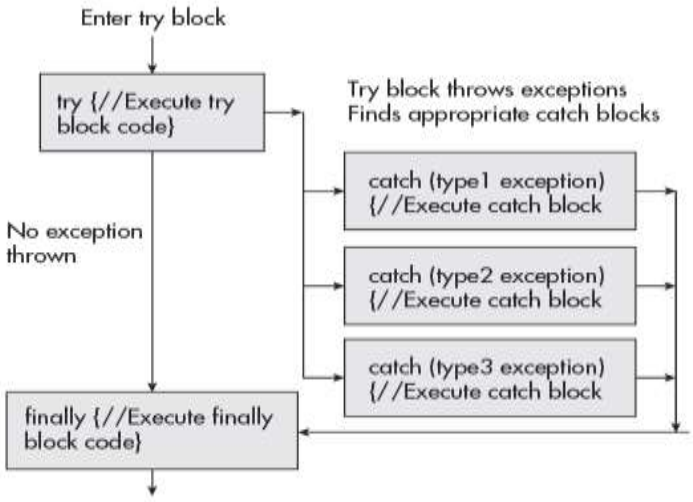
SURESH YADLAPATI



Using try and catch:

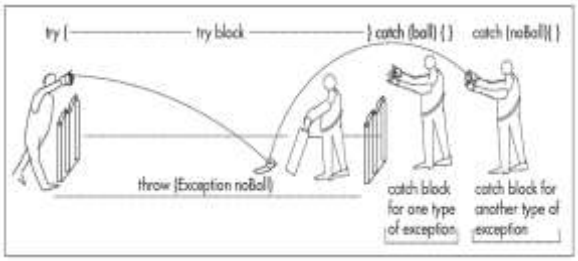
- The program code that is most likely to create exceptions is kept in the try block, which is followed by the catch block to handle the exception.
- In normal execution, the statements are executed and if there are no exceptions, the program flow goes to the code line after the catch blocks.
- However, if there is an exception, an exception object is thrown from the try block.
- Its data members keep the information about the *type* of exception thrown.
- The program flow comes out of the try block and searches for an appropriate catch block with the same *type* as its argument.

SURESH YADLAPATI



catch block

- A catch block is meant to catch the exception if the *type* of its argument matches with the *type* of exception thrown.
- If the *type* of exception does not match the *type* of the first catch block, the program flow checks the other catch blocks one by one.
- If the *type* of a catch block matches, its statements are executed.
- If none matches, the program flow records the *type* of exception, executes the finally block, and terminates the program.



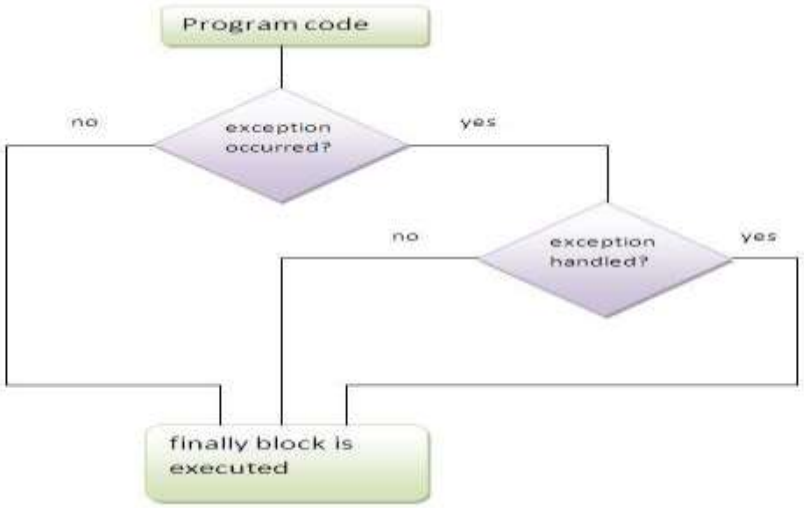
Example:

```
class UsingTry_Catch
{
        public static void main(String args[])
        {
                int d, a;
                try { // monitor a block of code.
                        d = 0;
                        a = 42 / d;
                        System.out.println("This will not be printed.");
                }
                catch (ArithmeticException e) { // catch divide-by-zero error
                        System.out.println("Division by zero.");
                }
                System.out.println("After catch statement.");
        }
}
```

SURESH YADLAPATI

finally block:

- Java finally block is a block that is used *to execute important code* such as closing connections (databases, network, disks, commit in databases) etc.
- Java finally block is always executed whether exception is handled or not.



Ex: Finally case1: Exception occurred and handled

```
class Finally_Case3   //exception occured and handled.
{
    public static void main(String args[])
    {
        try{
            int data=25/0;
            System.out.println(data);
        }
        catch(ArithmeticException e)
        {
                System.out.println(e);
        }
        finally
        {
                System.out.println("finally block is always executed");
        }
        System.out.println("rest of the code...");
     }
}
```

Ex: Finally case2: Exception occurred and not handled

```
class Finally_Case2   //exception occured and not handled.
{
          public static void main(String args[])
          {
              try{
                      int data=25/0;
                      System.out.println(data);
              }
```

SURESH YADLAPATI

```
                finally
                {
                        System.out.println("finally block is always executed");
                }
                System.out.println("rest of the code...");
        }
    }
```

Ex: Finally case3: Exception not occurred

```
class Finally_Case1
{
        public static void main(String args[])
        {
            try{
                int data=25/2;
                System.out.println(data);
            }
            catch(ArithmeticException e)
            {
                    System.out.println(e);
            }
            finally
            {
                    System.out.println("finally block is always executed");
            }
            System.out.println("rest of the code...");
        }
}
```

SURESH YADLAPATI

Uncaught exception:

In Java, whenever an exception is thrown, it must be caught by handlers. If no catch block is provided or an exception is not caught it is handle by JVM default handler and usually terminates the program.

```
class Uncaught
{
  public static void main(String ad[])
  {
                try{
                int d=0;
                int a=42/d;
                System.out.println(a);
                }
                finally{
                        System.out.println("finally block");
                }
  }
}
```

Output:

```
finally block
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Uncaught.main(Uncaught.java:7)
```

finalize in Exception handling:

- finalize() method releases system resources before the garbage collector runs for a specific object.
- JVM allows finalize() to be invoked only once per object

```
@Override
// protected void finalize() throws Throwable {
   try{ ……. }
   catch(Throwable t) { throw t; }
   finally{ super.finalize(); }
 }
```

Following points need to be taken into consideration when using finalize() method:

1. super.finalize() method is always called in finalize() method.
2. Time critical application logic should not be placed in finalize() method.

SURESH YADLAPATI

Multiple catch clauses:

- In some cases, more than one exception could be raised by a single piece of code.
- To handle this type of situation, you can specify two or more catch clauses, each catching a different type of exception.
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.
- If one catch statement is executed, the others are bypassed, and execution continues after the try / catch block.
- When you use multiple catch statements, it is important to remember that exception subclasses must come before any of their super classes.
- This is because a catch statement that uses a super class will catch exceptions of that type plus any of its subclasses. Subclass would never be reached if it came after its super class.
- Since Java 7, a modification has been done in the language so that one catch block can catch more than one type of exception.

- The exception classes are connected by Boolean | (or) operator |.

Ex:

```
class MultipleCatches
{
  public static void main(String args[])
  {
        try {
              int a = args.length;
              System.out.println("a = " + a);
              int b = 42 / a;
              int c[] = {1};
              c[42] = 99;
        }
        catch(ArrayIndexOutOfBoundsException  e)
        {
              System.out.println("Array index out of bounds: " + e);
        }
        catch(ArithmeticException e)
        {
              System.out.println("Arithemetic Exception: " + e);
        }
        catch(Exception e)
        {
              System.out.println(e);
        }
        System.out.println("After try/catch blocks.");
  }
}
```

SURESH YADLAPATI

Or else you can use the following code fragment after try block:

```
        catch(ArithmeticException | ArrayIndexOutOfBoundsException  e)
        {
              System.out.println("Exception : " + e);
        }
```

Ex: ArrayIndexOutofBounds Exception example program

```
// Demonstrate multiple catch statements.
class ExProgram
{
  public static void main(String args[])
  {
        try {
              int c[] = {1};
              c[42] = 99;
        }
        catch(ArrayIndexOutOfBoundsException  e)
```

```
            {
                    System.out.println("Array index out of bounds: " + e);
            }
            System.out.println("After try/catch blocks.");
            }
}
```

Use of throw clause:

- The throw keyword in Java is used to explicitly throw an exception from a method or any block of code. We can throw either checked or unchecked exception. The throw keyword is mainly used to throw custom exceptions.
- The general form of throw is shown here:

  throw ThrowableInstance;
- Here, ThrowableInstance must be an object of type Throwable or a subclass of Throwable.
- There are two ways to obtain a Throwable instance:
  - creating one with the new operator

    throw new exception_class("error message");
  - using the parameter of catch clause -

    throw exception_type_variable; (For rethrowing)
- The flow of execution stops immediately after the throw statement; any subsequent statements are not executed.
- The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of exception.
- If it does find a match, control is transferred to that statement.
- If no matching catch is found, then the default exception handler halts the program and prints the stack trace.

  Ex:

  /* In this example, we have created the validate method that takes integer value as a parameter. If the age is less than 18, we are throwing an Exception otherwise print a message welcome to vote.*/

```
import java.util.*;
public class UseofThrow
{
    static void validate(int age)
        {
            try{
```

SURESH YADLAPATI

```
                    if(age<18)
                        throw new   ArithmeticException("not eligible to cast the vote\n");
                    else
                        System.out.println("Welcome to participate in voting");
                }
                catch( ArithmeticException e)
                {
                    System.out.println(e);
                    throw e; //rethrowing an exception
                }
        }
        public static void main(String args[])
        {
            Scanner s=new Scanner(System.in);
            System.out.println("Enter Age");
            int age=s.nextInt();
            try{
                validate(age);
            }
            catch(Exception e)
            {
                System.out.println("Exception caught again:"+e);
            }
        }
}
```

SURESH YADLAPATI

Use of throws clause:

- throws is a keyword in Java that is used in the signature of a method to indicate that this method might throw one of the listed type exceptions(Checked or un-checked).
- The caller to these methods has to handle the exception using a try-catch block.

```
type method-name(parameter-list) throws exception-list
{
    // body of method
}
```

Ex:

```
class ThrowsDemo
{
    static void throwMethod() throws Exception
    {
            System.out.println ("Inside throwMethod");
            throw new Exception("Demo");
```

```
    }
    public static void main(String args[])
    {
      try{
                    throwMethod();
          }
          catch(Exception e)
          {
                System.out.println("Inside  main():"+e);
          }
    }
}
```

Class Throwable:

- The class Throwable is super class to all the error and exception classes.
- The class is declared as

  public class Throwable extends Object implements serializable
- The program code can throw only objects of this class or objects of its subclasses.
- Only the object of Throwable class or its subclasses can be the arguments of catch clause.
- It would be a bad programming practice to throw Throwable or make Throwable as argument of catch blocks because it will hide desirable information.

SURESH YADLAPATI

Constructors:

i. Public Constructors

Throwable(): It is a non-parameterized constructor which constructs a new Throwable with null as its detailed message.
Throwable(String message): It is a parameterized constructor which constructs a new Throwable with the specific detailed message.
Throwable(String message, Throwable cause): It is a parameterized constructor which constructs a new Throwable with the specific detailed message and a cause.
Throwable(Throwable cause): It is a parameterized constructor which constructs a new Throwable with the specific cause and a detailed message of the cause

ii. Protected constructors:

Throwable(String message, Throwable cause, boolean enableSuppression, boolean writableStackTrace):
The parameters are:-
message – the detail message.
cause – the cause
enableSuppression – whether or not suppression is enabled or disabled
writableStackTrace – whether or not the stack trace should be writable.

Methods:

Table 11.3 Commonly used methods defined in class `Throwable`

Method name	*Description*
`void addSuppressed( Throwable exception)`	The specified exception is appended to exceptions that were suppressed.
`Throwable fillInStackTrace()`	The method fills in execution stack trace.
`Throwable getCause()`	It returns the cause of the throwable, otherwise false if not available.
`String getMessage ()`	The method returns a detailed string message that gives information about the current exception.
`String getLocalizedMessage()`	It returns a string version of a localized description of this throwable.
`StackTraceElement[] getStackTrace()`	It gives access to StackTraceElement.
`Throwable [] getSuppressed()`	It returns an array of all exceptions suppressed by try-with-resources.
`Throwable initCause( Throwable cause)`	It initializes with specified value of the cause of this throwable.
`void printStackTrace()`	It traces and displays the hierarchy of method calls that resulted in exception.
`void printStackTrace( printStream strm)`	It prints this throwable and its back trace to standard error stream.
`void printStackTrace( printWriter strm)`	The method prints this throwable and the trace to the specified print writer.
`void setStackTrace( StackTraceElement[] stackTrace)`	The method sets the stack trace elements.
`String toString()`	The class Throwable overrides the method `toString()` of object class for displaying a short message about this throwable.

SURESH YADL

Unchecked exceptions:

- The unchecked exception classes are subclasses of class
- RuntimeException, which is further a subclass of class Exception.
- The runtime exceptions are not checked by the compiler.
- These exceptions form a subgroup of classes that are subclasses of class Exception.

Table 11.4 Unchecked subclasses of RuntimeException

Name of class	*Description of exception condition*
`ArithmeticException`	Exceptional arithmetic condition such as division by zero
`ArrayIndexOutOfBoundsException`	Exception arises when out of bound index number is used
`ArrayStoreException`	Exception arises when we try to use an object of wrong *type* in an array
`ClassCastException`	Exception condition arising out of invalid *type* object
`IllegalArgumentException`	Exception arises due to the use of an invalid argument in a method like placing a negative number as argument of sqrt()
`IllegalStateException`	Exception due to incorrect state of an application
`IllegalThreadStateException`	Exception arises when the operation attempted is not compatible with the thread state
`IndexOutOfBoundsException`	When index number used for array, vector, or string element is out of bounds
`NegativeArraySizeException`	Exception due to the use of negative number for size of array
`NumberFormatException`	Exception due to invalid conversion of string into number
`NullPointException`	Exception due to the use of object with null reference
`SecurityException`	When the code does an illegal operation that violates security
`TypeNotPresentException`	When the specified *type* is not found
`UnsupportedOperationException`	Exception due to unsupported operation

Checked exceptions: These are the exceptions that are checked at compile time. If some code within a method throws a checked exception, then the method must either handle the exception or it must specify the exception using the *throws* keyword.

Table 11.5 Checked Exceptions

Exception	*Description*
`ClassNotFoundException`	Exception because class is not found
`FileNotFoundException`	Exception if the file does not exist
`CloneNotSupportedException`	If an object does not implement Cloneable interface, an attempt to clone will result in this exception
`IllegalAccessException`	Attempt to access a class not permitted access
`InstantiationException`	Attempt to create objects of an abstract class or of an interface will result in this exception
`InterruptedException`	When one thread is interrupted by another thread
`NoSuchFieldException`	Request for a nonexistent field
`NoSuchMethodException`	Request for a nonexistent method

Example programs on checked exceptions:

Ex1: Null pointer exception

```
public class NullPointerExceptionEx{
        public static void main(String[] args) {
                int[] data = null;
                int len = data[2];
        }
}
```

Ex2: IO Exception

```
import java.io.*;;

public class IOExceptionExample {

   public static void main(String[] args) {

       try{

        BufferedReader r  = new BufferedReader(new FileReader("example.txt"));

        String line = r.readLine();

        System.out.println(line);

        r .close();

                }

                catch(Exception e){}

   }

}
```

SURESH YADLAPATI

Ex3: Class not found exception

```
class ClassNFException {

   // Main driver method
   public static void main(String[] args) throws ClassNotFoundException
   {
            // Calling the class gfg which is not present in the
            // current class temp instance of calling class
            Class temp = Class.forName("gfg");

            // It will throw ClassNotFoundException
   }
}
```

Ex4: Class cast exception

```
public class ClassCastExceptionDemo
{
   public static void main(String[] args)
   {
      Object o = new String();
      Integer i = (Integer) o;
   }
}
```

Ex5. Interrupted exception

```
public class InterruptedExceptionExample {
   public static void main(String[] args) {
      Thread thread = new Thread(new Task());
      thread.start();

      try {
         // Main thread sleeps for 1 second
         Thread.sleep(1000);
         System.out.println("Main thread waking up and interrupting the other
thread...");
         // Interrupt the other thread
         thread.interrupt();
      } catch (InterruptedException e) {
         System.out.println("Main thread was interrupted.");
      }
   }
}

class Task implements Runnable {
   @Override
   public void run() {
      try {
         System.out.println("Task started, going to sleep for 5 seconds...");
         Thread.sleep(5000);  // Sleep for 5 seconds
         System.out.println("Task completed without interruption.");
      } catch (InterruptedException e) {
         System.out.println("Task was interrupted during sleep.");
      }
   }
}
```

SURESH YADLAPATI

Custom or User defined exceptions:

- We can also create our own exception by creating a sub class simply by extending java **Exception** class.

  Syntax:

  class UserDefinedException extends Exception

- Define a constructor in **UserDefinedException** (optional)
  - **UserDefinedException**();
  - **UserDefinedException** (parameter);

  The first form creates an exception that has no description. The second form lets you specify a description of the exception

  Ex:

```
import java.util.*;
class InvalidAgeException extends Exception
{
          InvalidAgeException()
          {
                    super("Age is lessthan 18 to cast the vote " );
          }
}
class UserDefinedException
{
    public static void main(String args[]){
     try{
          Scanner s=new Scanner(System.in);
          System.out.println("Enter Age to cast the vote");
          int age=s.nextInt();

              if(age<18)
                        throw new InvalidAgeException();
              else
                   System.out.println("Eligible to cast the vote");
         }
          catch(InvalidAgeException m)
          {
                        System.out.println("Exception occured :"+m);
          }
          System.out.println("rest of the code...");
  }
}
```

SURESH YADLAPATI